Università di Genova

Embedded Systems

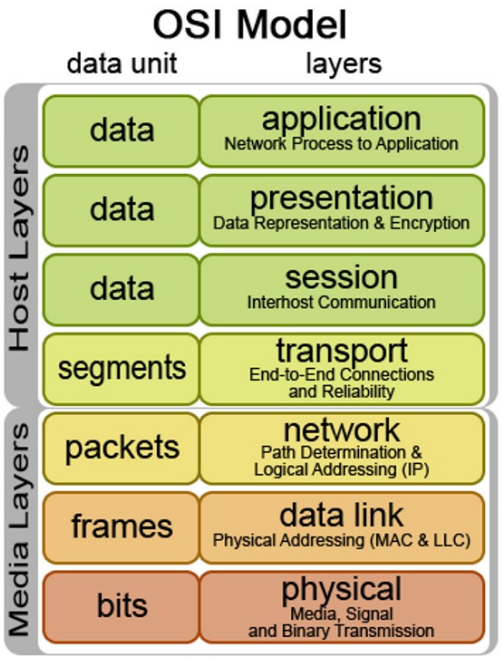
UART

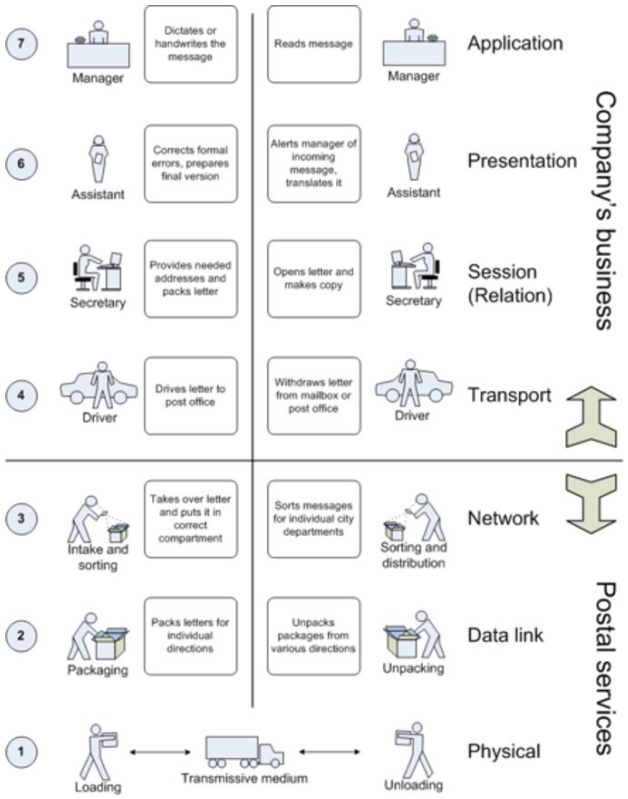
Prof. Enrico Simetti

Università di Genova

Introduction to Communications

ISO/OSI Model

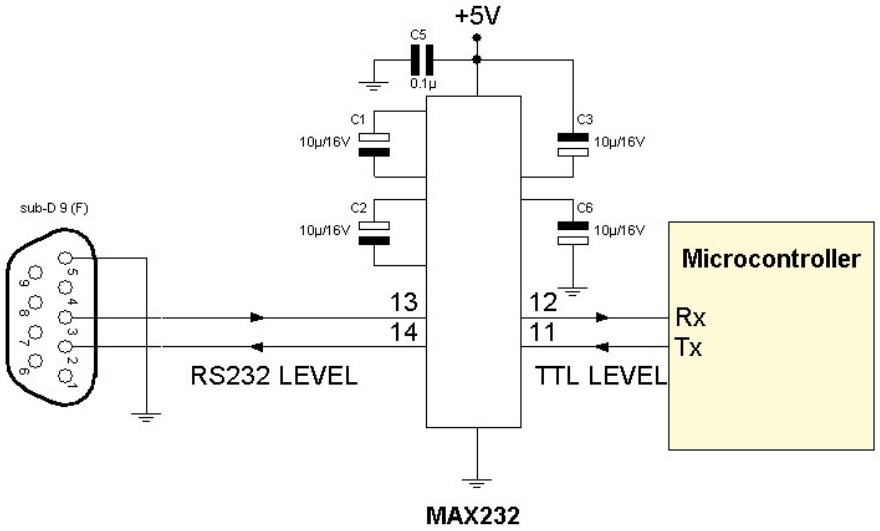




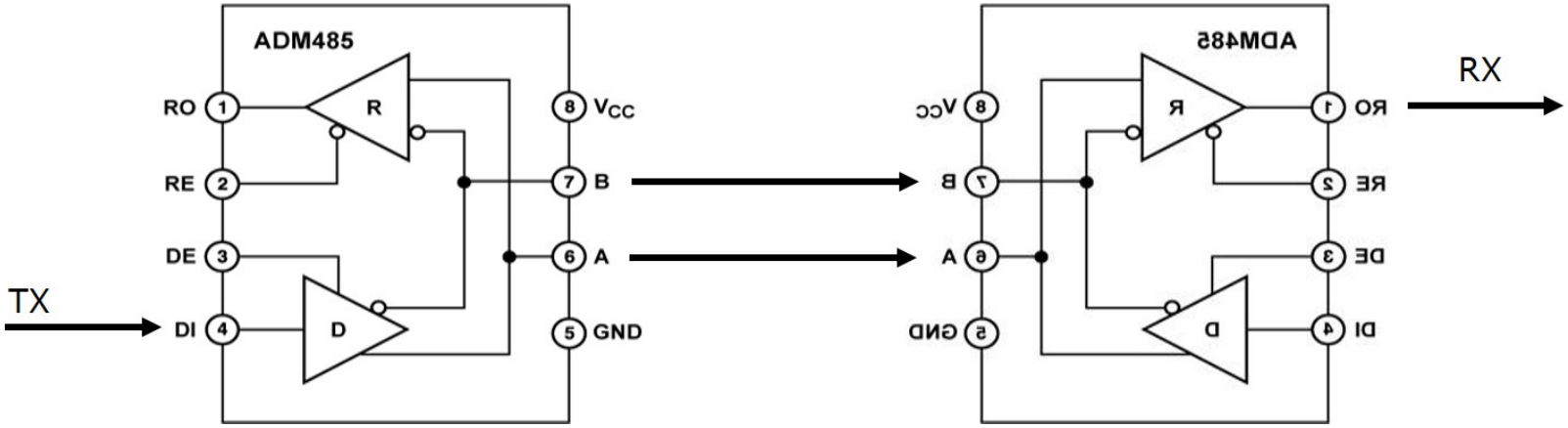
RM – OSI and letter communication parallel

Line Drivers

Line Driver: an amplifier used to improve the strength of an analog or digital signal at its source by driving the input to the transmission line with a higher-than-normal signal level. This increases the quality of a transmission over a long run of cable

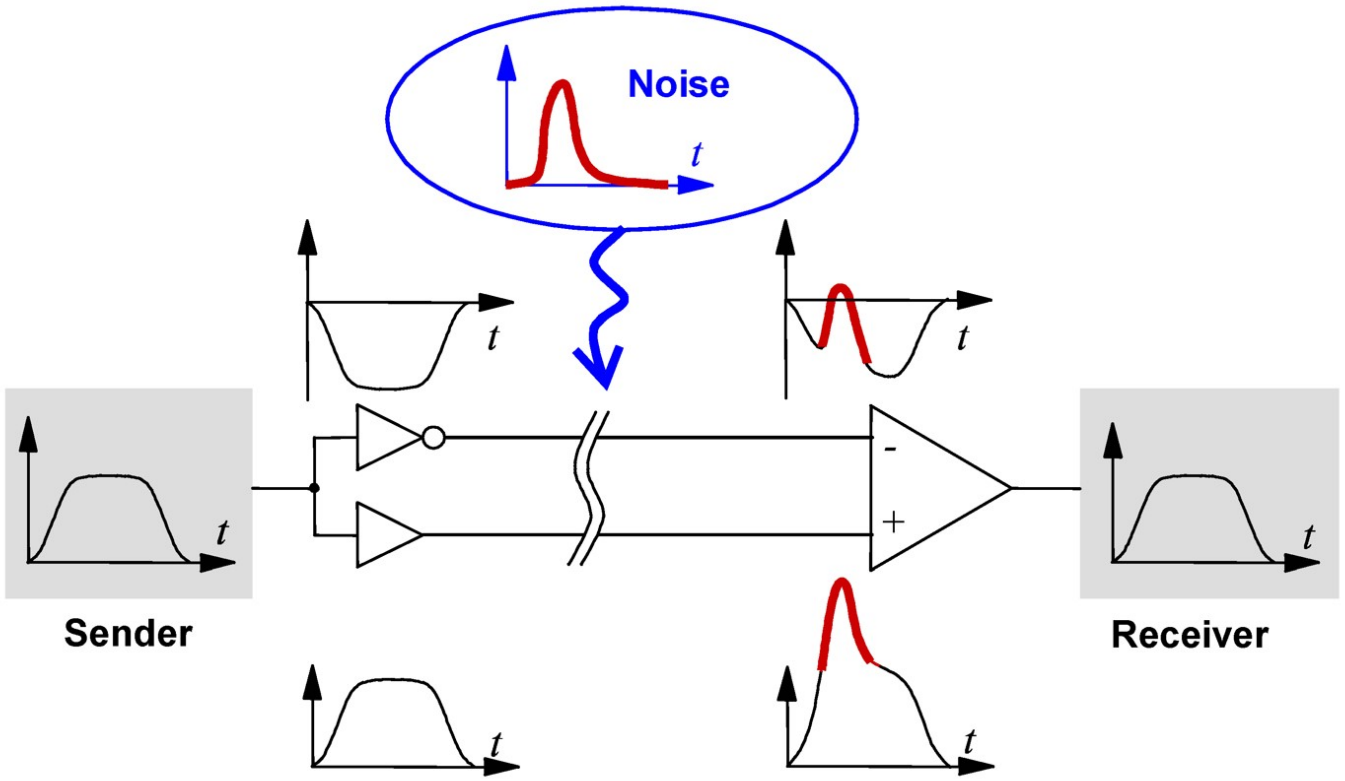


Differential Signaling

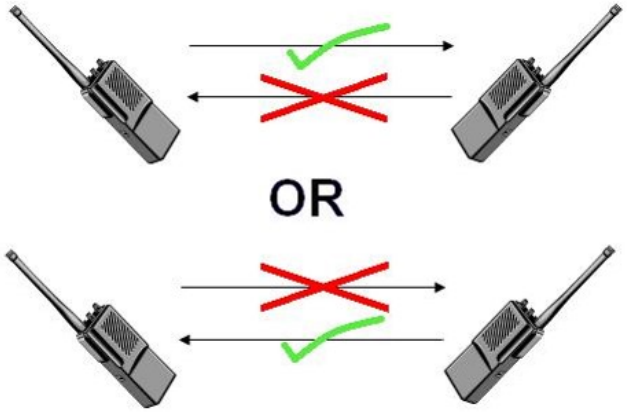


Differential signaling is a method for electrically transmitting information using two complementary signals. The receiving circuit responds to the electrical difference between the two signals, rather than the difference between a single wire and ground.

Differential Signaling



Type of Communications: Half/Full Duplex



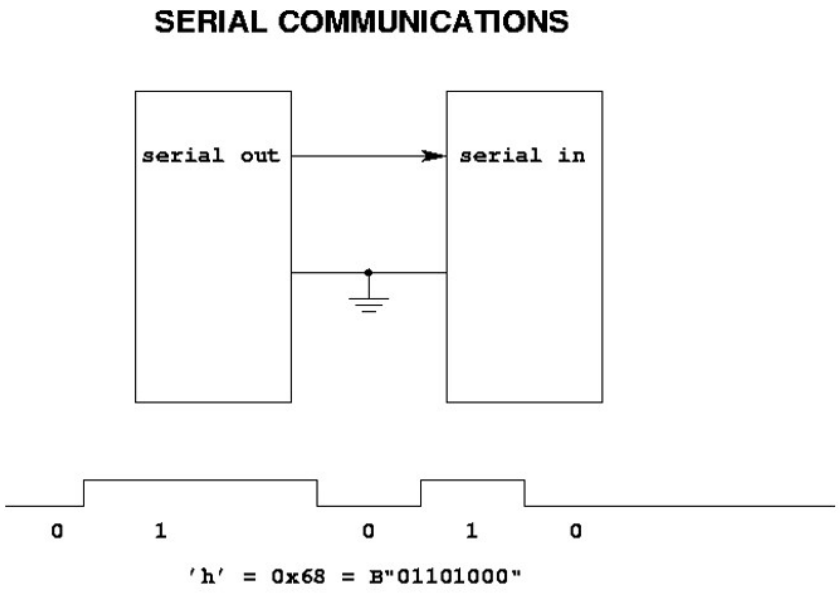
A half-duplex system provides for communication in both directions, but only one direction at a time (not simultaneously).

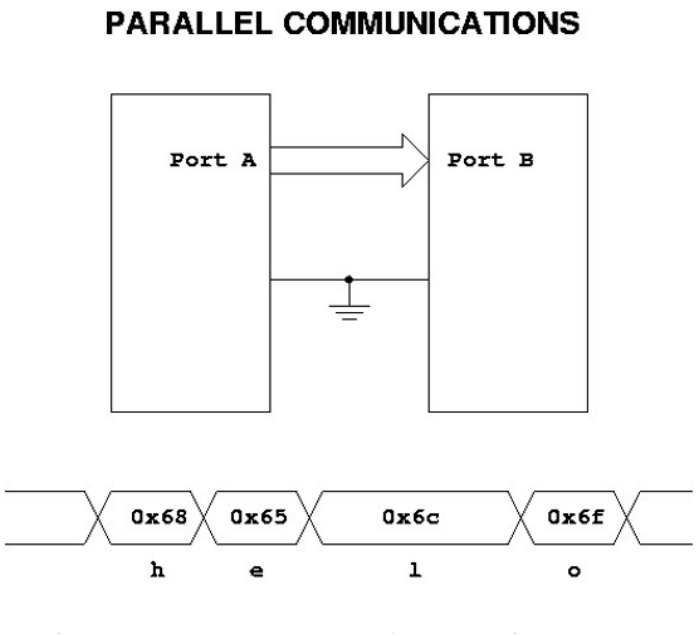
Type of Communications: Half/Full Duplex



A full-duplex, or sometimes double-duplex system allows communication in both directions, and unlike half-duplex, allows this to happen simultaneously.

Type of Communications: Sync vs Async





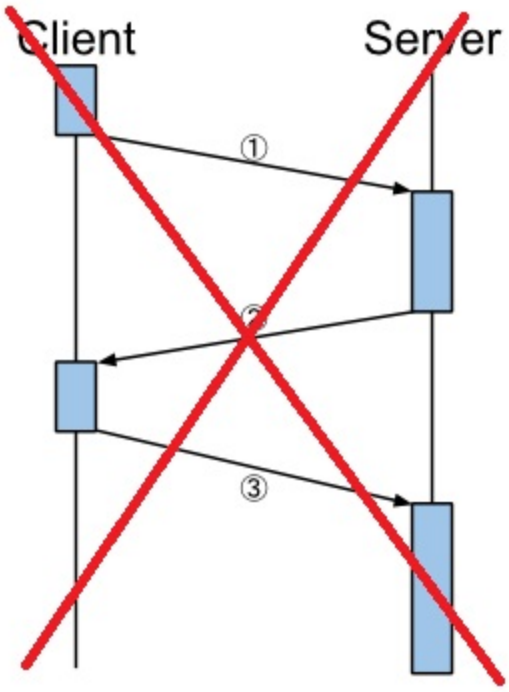
Type of Communications: Sync vs Async

Synchronous vs Asynchronous communication

- The difference lies in how the receiver understand there is a transmission on-going and when it should sample the data line

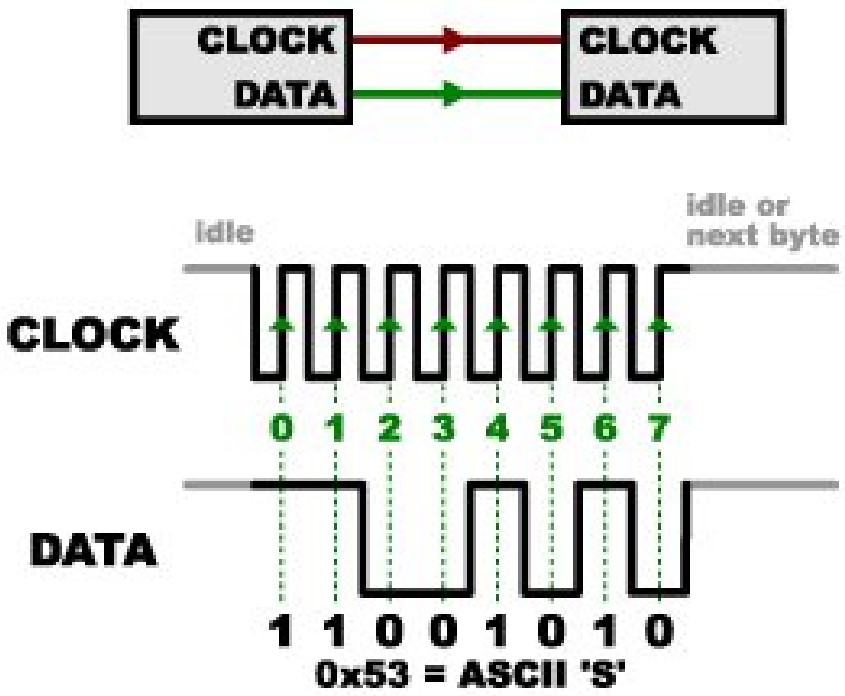
Type of Communications: Sync vs Async

This is not what we mean by synchronous communication in this context!



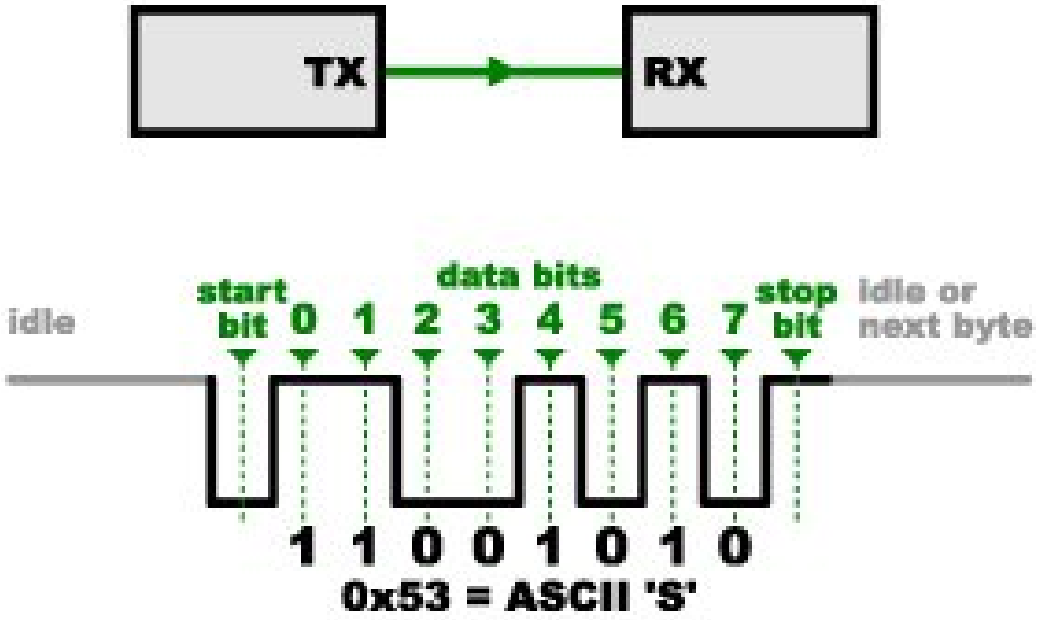
Type of Communications: Synchronous

Synchronous communication (e.g. SPI)



Type of Communications: Asynchronous

Asynchronous communication (e.g. UART)



Università di Genova

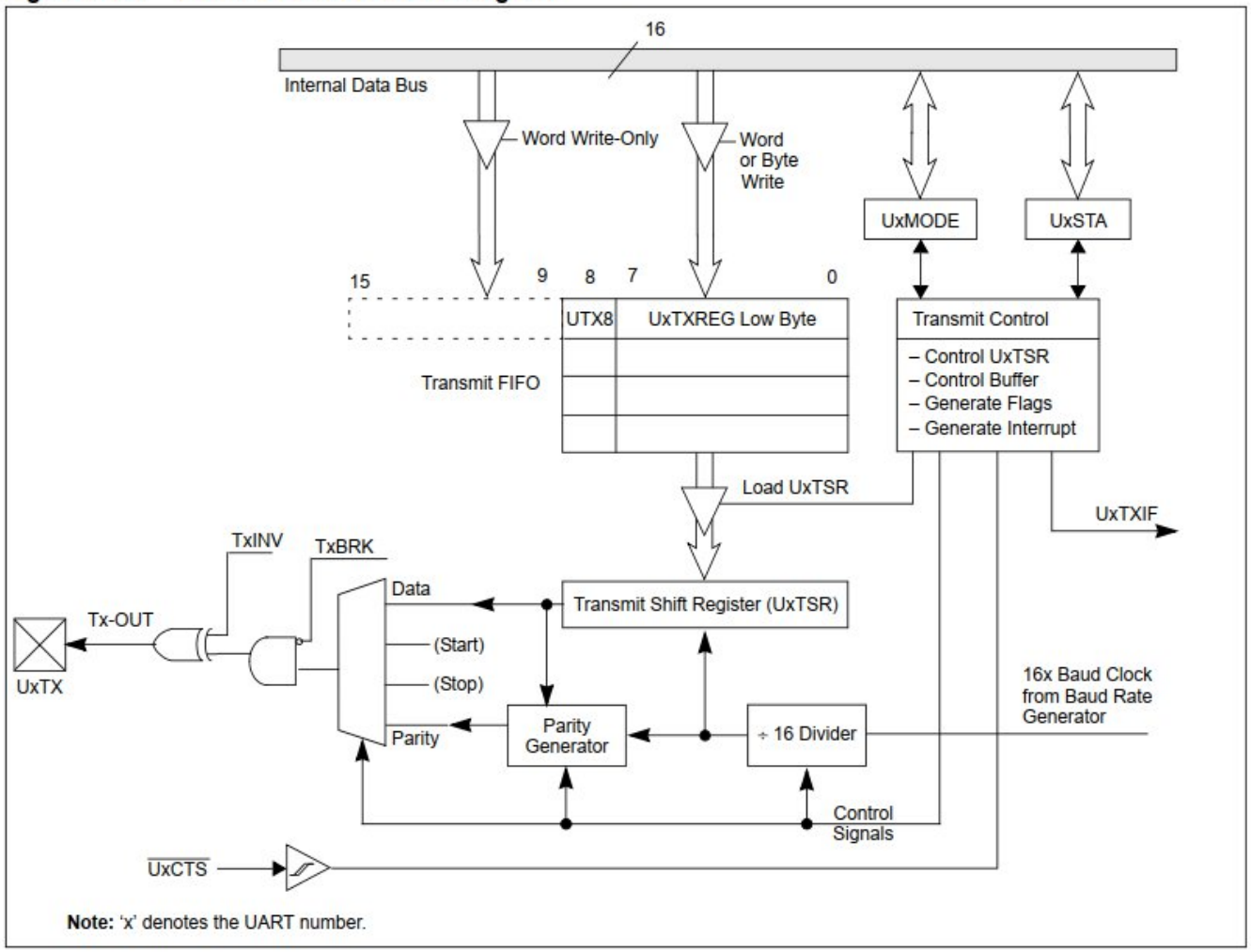
UART

dsPIC33F UART

- Full-duplex 8- or 9-bit data transmission
- Even, Odd or No Parity options (for 8-bit data)
- One or two Stop bits
- Fully integrated Baud Rate Generator with 16-bit prescaler
- Baud rates ranging from 38 bps to 10 Mbps at FCY = 40 MHz
- 4-deep First-In-First-Out (FIFO) transmit data buffer
- 4-deep FIFO receive data buffer
- Parity, Framing and Buffer Overrun error detection
- Support for 9-bit mode with Address Detect (9th bit = 1)
- Transmit and Receive Interrupts
- Loopback mode for diagnostic support

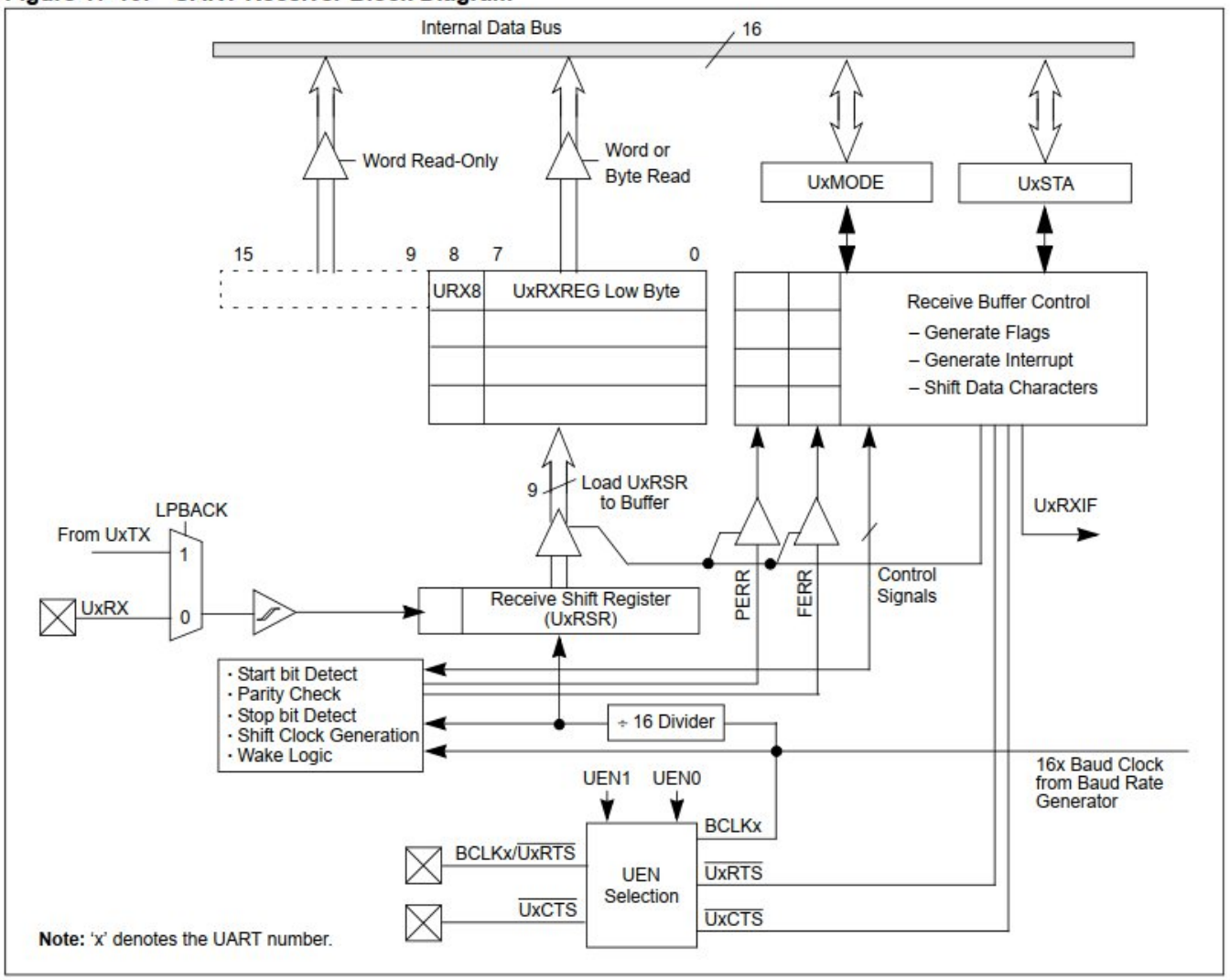
UART Transmitter

Figure 17-3: UART Transmitter Block Diagram



UART Receiver

Figure 17-10: UART Receiver Block Diagram



UART Status Register

REGISTER 20-2: UxSTA: UARTx STATUS AND CONTROL REGISTER

R/W-0	R/W-0	R/W-0	U-0	R/W-0, HC	R/W-0	R-0	R-1
UTXISEL1	UTXINV	UTXISEL0	—	UTXBRK	UTXEN(1)	UTXBF	TRMT
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R-1	R-0	R-0	R/C-0	R-0
URXISEL<1:0>		ADDEN	RIDLE	PERR	FERR	OERR	URXDA
bit 7							bit 0

UxSTA: UART status registers, contains the flags that notify the program of certain events.

- UTXISEL: selects when the IF is set to 1 during transmission
- URXISEL: selects when the IF is set to 1 during reception
- UTXEN: enables transmission
- TRMT: notifies if there is an on-going transmission
- URXDA: notifies if there are characters to be read
- OERR: notifies if an overflow error occurred

Managing Overflows

After an overflow occurred, and UxSTAbits.OERR = 1, the peripheral will discard any incoming bytes, until the user resets

UxSTAbits.OERR = 0;

We have the following choices when an overflow occurred:

- read all the bytes in the UART buffer, then clear OERR;
- clear OERR immediately (the bytes are lost);

Interrupt Selection Bits

bit 15 **UTXISEL:** Transmission Interrupt Mode Selection bit
1 = Interrupt when a character is transferred to the Transmit Shift register and as result, the transmit buffer becomes empty
0 = Interrupt when a character is transferred to the Transmit Shift register (this implies that there is at least one character open in the transmit buffer)

bit 7-6 **URXISEL<1:0>:** Receive Interrupt Mode Selection bit
11 = Interrupt flag bit is set when Receive Buffer is full (i.e., has 4 data characters)
10 = Interrupt flag bit is set when Receive Buffer is 3/4 full (i.e., has 3 data characters)
0x = Interrupt flag bit is set when a character is received

Changing these bits influences when U1TXIF and U1RXIF go to 1, i.e.

about which kind of event the peripheral will alert us

UTXISEL and URXISEL do not enable the interrupts!

Check IEC0 register to enable the interrupts.

UART Mode Register

Register 17-1: UxMODE: UARTx Mode Register

R/W-0	U-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0	R/W-0
UARTEN	—	USIDL	IREN(1)	RTSMD	—	UEN<1:0>	
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
WAKE	LPBACK	ABAUD	URXINV	BRGH	PDSEL<1:0>		STSEL
bit 7							bit 0

UxMODE: UARTx Mode register.

- **UARTEN**: enables the UART module
- **PDSEL**: selects the parity and data bits

Other UART Registers

UxRXREG: contains the character received

UxTXREG: write here the character to be sent to the transmit buffer

UxBRG: the baud rate register

$$Baud\ Rate = \frac{FCY}{16 \bullet (UxBRG + 1)}$$

$$UxBRG = \frac{FCY}{16 \bullet Baud\ Rate} - 1$$

Note: FCY denotes the instruction cycle clock frequency.

Example UART Programming

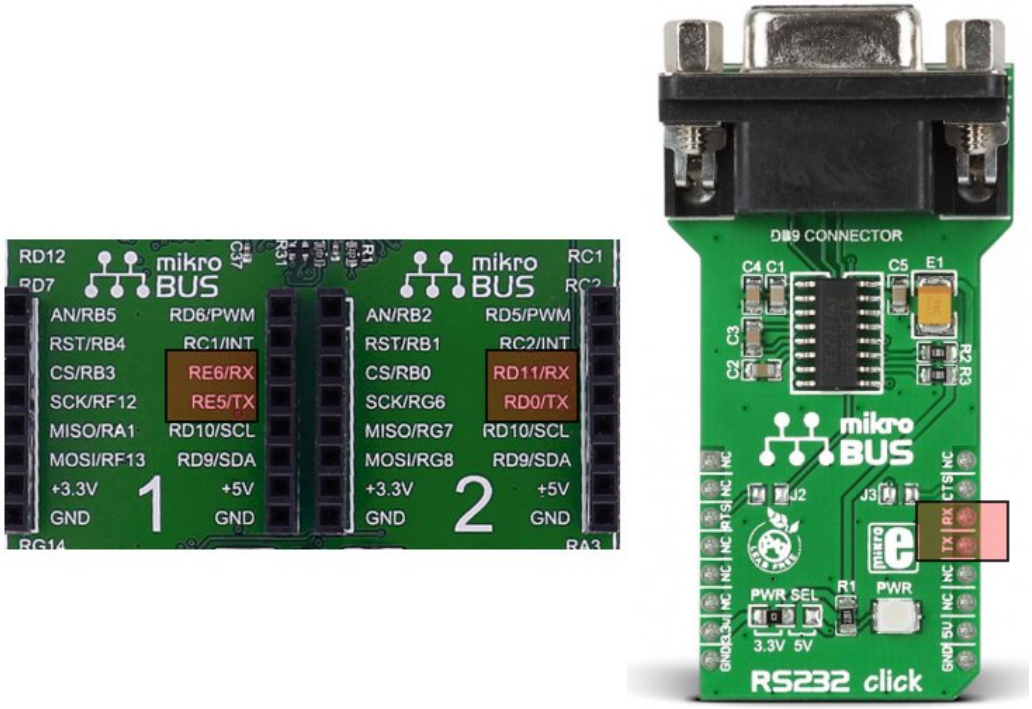
Example: sending one character at 9600 with UART1, assuming

7.3728 MHz oscillator (Fcy = 1.8432 MHz)

```
U1BRG = 11; // (7372800 / 4) / (16 * 9600)− 1
U1MODEbits.UARTEN = 1; // enable UART
U1STAbits.UTXEN = 1; // enable U1TX (must be after UARTEN)
U1TXREG = 'C'; // send 'C'
```

Clicker 2 Board UART

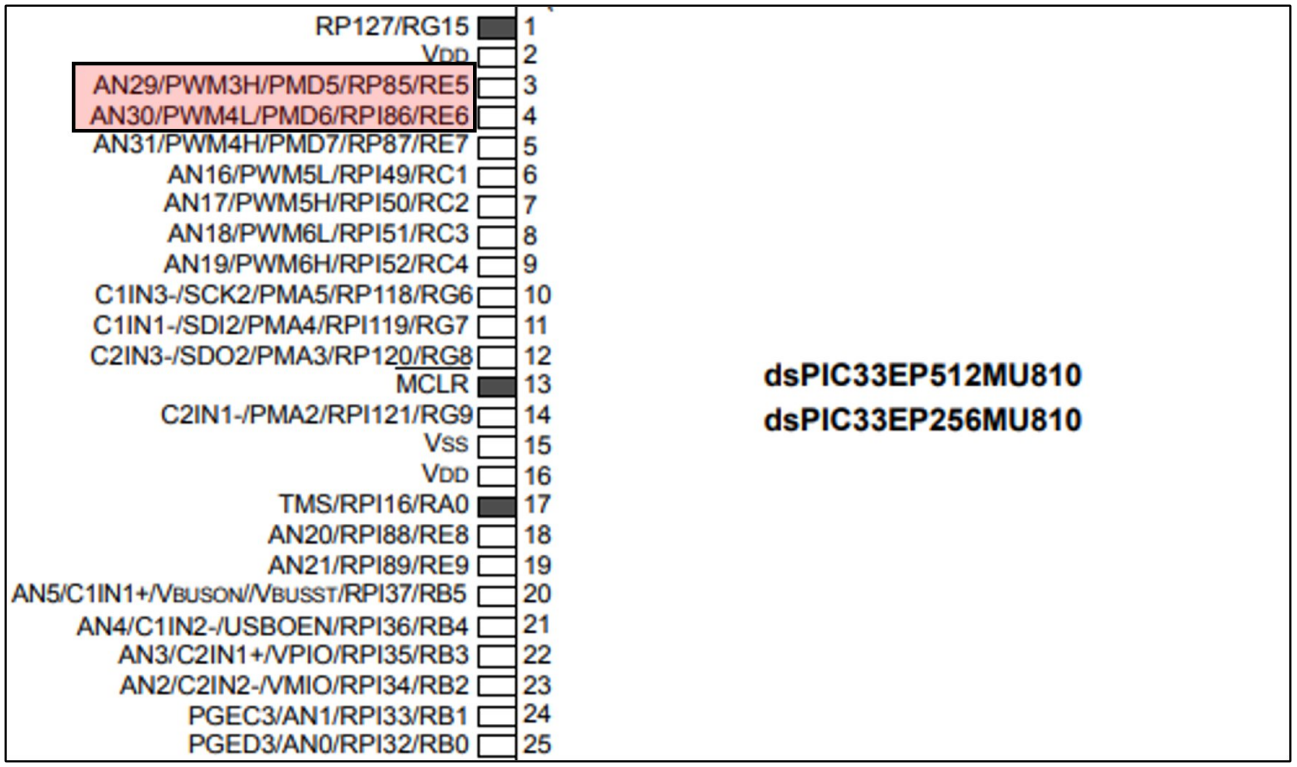
You must use the MikroBUS UART add-on board, inserted on one of the two on-board sockets:



Here we need to remap both input (RX) and output (TX)

Clicker 2 Board UART Pins

Example: UART pins remap on MikroBUS 1



Input pin can be remapped as shown in the Interrupts section

UART PPS Pin Remapping

TABLE 11-3: OUTPUT SELECTION FOR REMAPPABLE PINS (RPn)

Function	RPnR<5:0>	Output Name
DEFAULT PORT	000000	RPn tied to Default Pin
U1TX	000001	RPn tied to UART1 Transmit
U1RTS	000010	RPn tied to UART1 Ready-to-Send
U2TX	000011	RPn tied to UART2 Transmit
U2RTS	000100	RPn tied to UART2 Ready-to-Send
SDO1	000101	RPn tied to SPI1 Data Output
SCK1	000110	RPn tied to SPI1 Clock Output
SS1	000111	RPn tied to SPI1 Slave Select
SS2	001010	RPn tied to SPI2 Slave Select
CSDO	001011	RPn tied to DCI Data Output
CSCK	001100	RPn tied to DCI Clock Output
COFS	001101	RPn tied to DCI FSYNC Output
C1TX	001110	RPn tied to CAN1 Transmit
C2TX	001111	RPn tied to CAN2 Transmit
OC1	010000	RPn tied to Output Compare 1 Output
OC2	010001	RPn tied to Output Compare 2 Output
OC3	010010	RPn tied to Output Compare 3 Output
OC4	010011	RPn tied to Output Compare 4 Output
OC5	010100	RPn tied to Output Compare 5 Output
OC6	010101	RPn tied to Output Compare 6 Output
OC7	010110	RPn tied to Output Compare 7 Output
OC8	010111	RPn tied to Output Compare 8 Output
C1OUT	011000	RPn tied to Comparator Output 1
C2OUT	011001	RPn tied to Comparator Output 2
C3OUT	011010	RPn tied to Comparator Output 3
U3TX	011011	RPn tied to UART3 Transmit
U3RTS	011100	RPn tied to UART3 Ready-to-Send

Note 1: This function is available in dsPIC33EPXXX(MC/MU)806/810/814 devices only.

Assignment (basic)

1/ Configure the UART1 to use the MikroBUS 2.

2/ First find out the remappable I/O pins corresponding to the RD11 and RD0 registers, then perform the remapping of UART1.

3/ Configure the UART1 to work at 9600 bps.

4/ Read a character from UART1 and send it back to the UART1

Assignment (advanced)

1/ Simulate a function call that needs 7 ms for its execution and needs to work at 100 Hz.

2/ Make LD2 blink at 2.5 Hz.

3/ Read characters from UART1. Keep a counter of how many chars you have received. If a string “LD1” is received, toggle the status of LD1. If a string “LD2” is received stop or resume the blinking of LD2.

4/ Whenever button T2 is pressed, send “C=xx”, where xx is the current number of chars received to the UART1

5/ Whenever button T3 is pressed, send “D=yy”, where yy is the number of times the deadline of the tmr_wait_period() was missed.

6/ Follow the template provided in the next slide.

Template Code

```
void algorithm() {
  tmr_wait_ms(TIMER2, 7);
}
int main() {
  tmr_setup_period(TIMER1, 10);
  while(1) {
    algorithm();
    // code to handle the assignment
    ret = tmr_wait_period(TIMER1);
  }
}
```



Università
di Genova